Unit 9 Using Shell Variables

Course code LX13
Linux Basics

Objectives

After completing this unit, students should be able to:

- List variables that define your environment
- Set, reference and delete variables
- Define the use of the following quoting metacharacters:
  - Double quotes "
  - Single quotes '
  - Backslash \
- Perform command substitution
  - Back quotes `
  - Parenthesis $( )

Shell Variables

- Variables are names known to a program that represent data whose value may change at any time while the program is running

- Shell variables define your environment:
  HOME directory
  TERMinal type
  Search **PATH**

- Additional variables can be defined

Listing Variable Settings

- One way to find out what variables are set in your environmnet is to use the `set` command.

```
$ set
HOME=/home/tux1
PATH=/usr/bin:/usr/local/bin:/bin:/usr/X11R6/bin:/home/tux1/bin
PS1=[\u@\h \W]\$
PS2=>
SHELL=/bin/bash
TERM=linux
xy=day
$
```

Special Shell Variables

- **$TERM**: Terminal type
- **$PATH**: Search path for executables
- **$MANPATH**: Search path for manual pages
- **$HOME**: Location of your home directory
- **$LANG**: Language to be used
- **$PS1, $PS2**: Various prompts
- **$SHELL**: Name of your current shell

Setting and Referencing Shell Variables

- To assign a value to a shell variable

```
$ name=value
```

- To reference a variable, prefix its name with a dollar sign

```
$ xy="Hello world"
$ echo $xy
Hello world
```

- To delete a variable, use the **unset** command

```
$ unset xy
$ echo $xy

$
```

Shell Variable Example

```
$ xy=day
$ echo $xy
day

$ echo Tomorrow is Tues$xy
Tomorrow is Tuesday

$ echo There will be a $xylong meeting
There will be a meeting

$ echo There will be a ${xy}long meeting
There will a daylong meeting
```

Command Substitution

- The value of a variable can be set to the output from a command or group of commands.

```
$ date
Tue Apr  6  11:30:34  CEST  1999
```

```
$ now=`date`
$ echo $now
Tue Apr  6  11:30:34  CEST  1999
```

(sometime later ...)

```
$ echo $now
Tue Apr  6  11:30:34  CEST  1999
```

Parameter Substitution

- **xargs** reads parameters from stdin and executes a certain command with those parameters

```
$ ls | xargs lpr
-equals-
$ lpr `ls`
```

- But... xargs can work with an unlimited number of parameters by calling **lpr** multiple times
- You can even limit the number of arguments:

```
$ ls | xargs -n 2 echo two files:
$ ls | xargs -n 1 echo one file:
```

Quoting Metacharacters

- **Single Quotes**

```
$ echo '$xy * Text'
$xy * Text
```

Ignore **all** metacharacters

- **Double Quotes**

```
$ echo "$xy * Text"
day * Text
```

Ignore all metacharacters except for **$** **`** and ****

- **Backslash**

```
$ echo \$xy \*  \Text
$xy * Text
```

Ignore the special meaning of the **following** character

Command Line Parsing

- Piping `|`
- Redirection `> < >> 2>`
- Command and variable substitution `$var` `` `command` ``
- Wildcard expansion `? * [ ]`
- Command execution

Checkpoint

What is the output of

1. $ echo "Files in the current directory are *"

---

2. $ echo *

---

3. $ echo "Who is on the system: `who`"

Unit Summary

- The shell has built-in variables which define your environment and lets you define variables of your own

- Variables can be set to a value which can then be referenced and used within scripts

- The following shell metacharacters have been discussed:

  ' string'
  "string"
  \character
  `command`